

Module: Les technologies du Middleware



Cours2: Intergiciels à appel de procédure
distante (DCE) (RPC Middlewares)



- Modèle Client/serveur
- RPC: Principe
- RPC : Fonctionnement
- RPC : Exemples d'implémentation



Critères de classification:

■ Nature des entités qui communiquent

- Objets
- Composants
- Autres

■ Mode d'accès aux services

- Synchrone (client-serveur)
- Asynchrone (événements, messages)
- Mixte

■ Autres critères

- Entités fixes / mobiles
- Performances / qualité de service garanties ou non

Pas de classification rigoureuse en raison de la diversité des réalisations



- **Modèle Client-Serveur :**

 - RPC, RMI, CORBA, Servlet,...

- **Modèle de communication par messages :**

 - MOM : Message Oriented Middleware,
Files de messages.

- **Modèle de communication par événements.**

- **Modèle à base de composants :**

 - Bean, EJB.

- **Modèle à base d'agents mobiles :**

 - Agglet, Voyager.

- **Modèles à mémoires « virtuelles » partagées :**

 - Modèles à espace de tuples.

 - Modèles à objets dupliqués.



Coté serveur :

- ✚ Externalisation de services.
- ✚ Attente de requêtes en provenances de clients puis exécution des
- ✚ requêtes en séquentiel ou en parallèle.
- ✚ Interface : **Skeleton**
 - ✚ reçoit l'appel sous forme de « stream »,
 - ✚ décapsule les paramètres,
 - ✚ demande l'exécution,
 - ✚ renvoi les paramètres (par références) et les résultats.

Coté client :

- ✚ Envoi de requêtes puis attente de la réponse,
- ✚ Initiateur du dialogue,
- ✚ Interface : **Stub**
 - ✚ reçoit l'appel en local,
 - ✚ encapsule les paramètres,
 - ✚ attends les résultats du serveur,
 - ✚ décapsule les résultats,
 - ✚ redonne la main à la fonction appelante.



- **L'appel de procédure à distance (DCE) ou l' appel RPC**
(pour *remote procedure call*) est un outil pour construire des applications répartie de type « client-serveur »



■ Mode de communication entre deux programmes distants

- Un programme jouera le rôle de client
- L'autre programme jouera le rôle de serveur

Le but de l'interaction du client avec le serveur:

- ✓ le client peut faire exécuter à distance une procédure par le serveur.
- ✓ Service basique (API d'appel de procédure distante, bibliothèque système)
 - Côté client : **invoque l'appel distant et récupère le résultat**
invoque (id_client, id_serveur, nom_procedure, paramètres);
 - Côté client : on invoque une procédure localisée à distance (Orienté objet)
ref_objet_serveur.nom_procedure (paramètres);
 - Côté serveur : reçoit, traite un appel et répond
traite (id_client, id_serveur, nom_procedure, paramètres);
 - Côté serveur : on déploie l'objet qui implante la procédure (Orienté objet)
method nom_procedure (paramètres);
- ✓ Service niveau langage (API définie au niveau du langage de programmation)

- **S'affranchir du côté basique des communications en mode message**
 - Ne pas avoir à programmer des échanges au niveau réseau en mode message.
- **Utiliser une structure familière: l'appel de procédure**
 - Problème: ne pas ignorer les différences centralisé/réparti.
- **Disposer de mécanismes modernes de programmation**
 - Vision modulaire des applications réparties (en approche objets répartis ou par composants sur étagères).
 - Réutilisation en univers réparti.



- **Les approches à RPC traditionnelles**

- SUN RPC

- **Approches à RPC intégrées dans les systèmes d'objets répartis**

- OMG CORBA (Object Management Group - Common Object Request Broker Architecture)

- SUN Java RMI

- Microsoft .NET

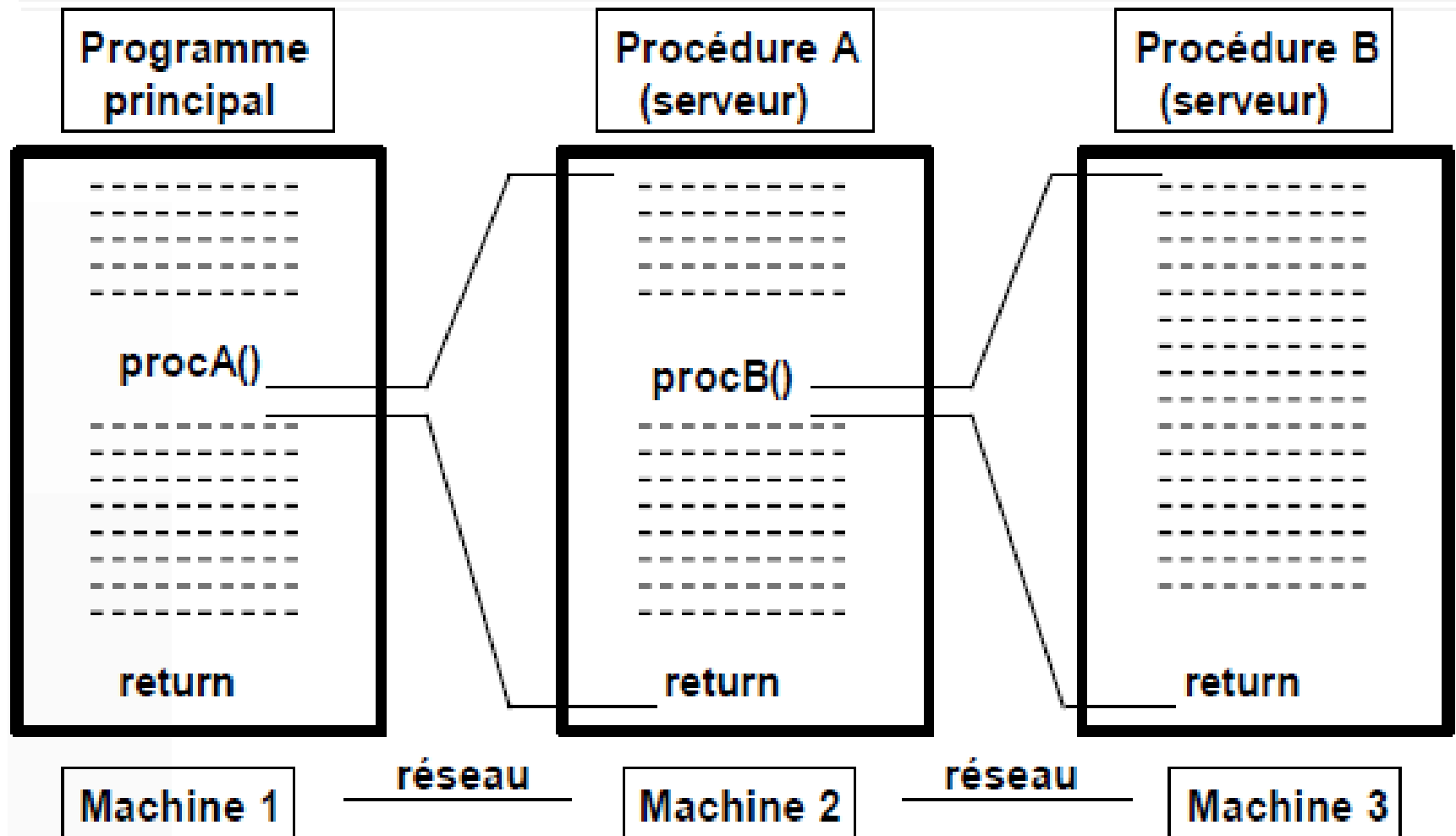
- **Approches à RPC intégrées dans les systèmes de composants**

- SUN J2EE EJB: Java 2 (Platform) Enterprise Edition – Enterprise Java Beans

- OMG CCM: Object Management Group - Corba Component Model

- WS-SOAP: Web Services - Simple Object Access Protocol

Client/Serveur : RPC





RPC: Principe (1)

Client

Server

appelant

appelé

protocol

```
int foo(int x, int y) {  
    if (x>100)  
        return(y-2);  
    else if (x>10)  
        return(y-x);  
    else  
        return(x+y);  
}
```

Appeler
sous-programme

Emballer
paramètres d'entrée

Attendre réponse

Déballer
paramètres de sortie

Envoyer paramètres d'entrée

Allouer processus

Déterminer
sous-programme

Exécuter localement
sous-programme

Emballer
paramètres de sortie

Envoyer paramètres de sortie

Désallouer processus

temps

temps

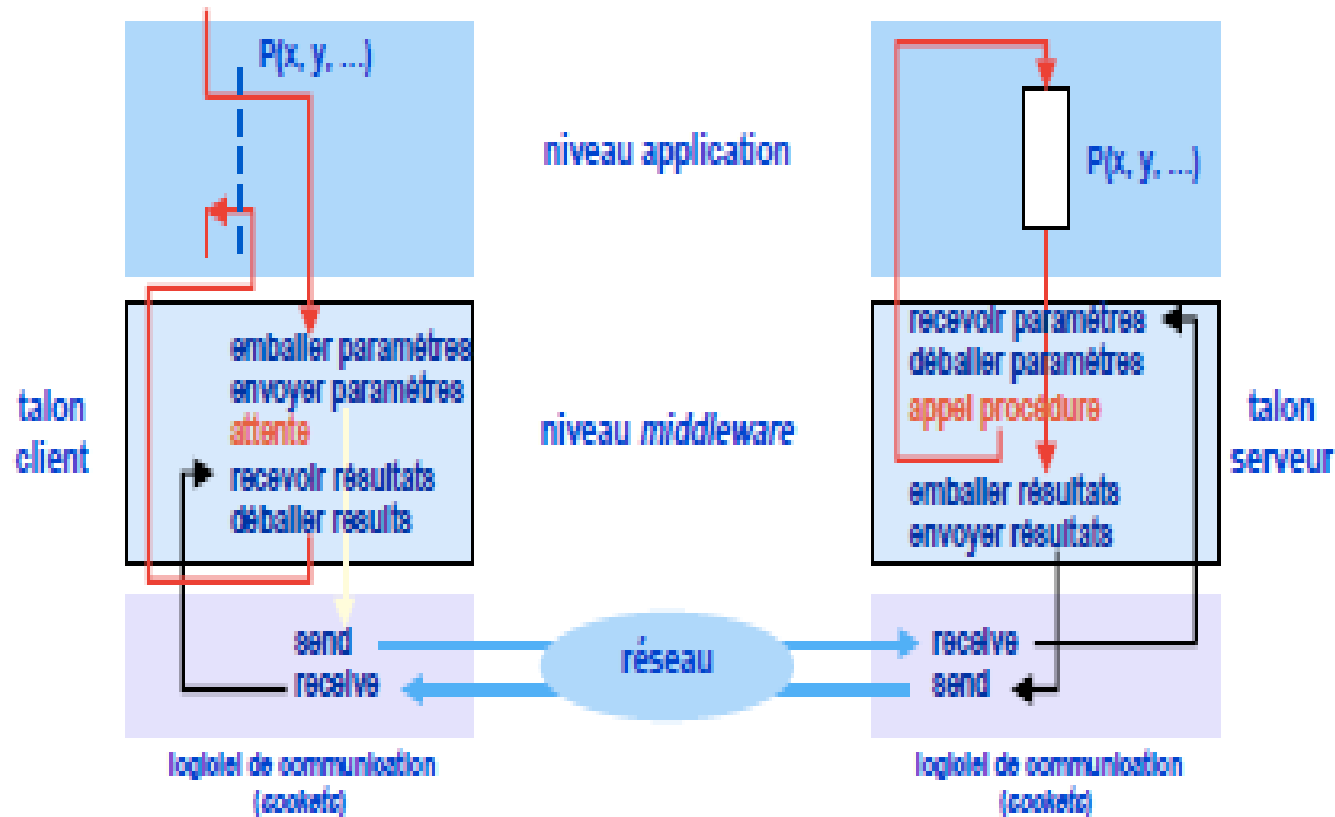
blah, blah, blah

bar = foo(a,b);

blah, blah, blah



RPC: Principe (2)

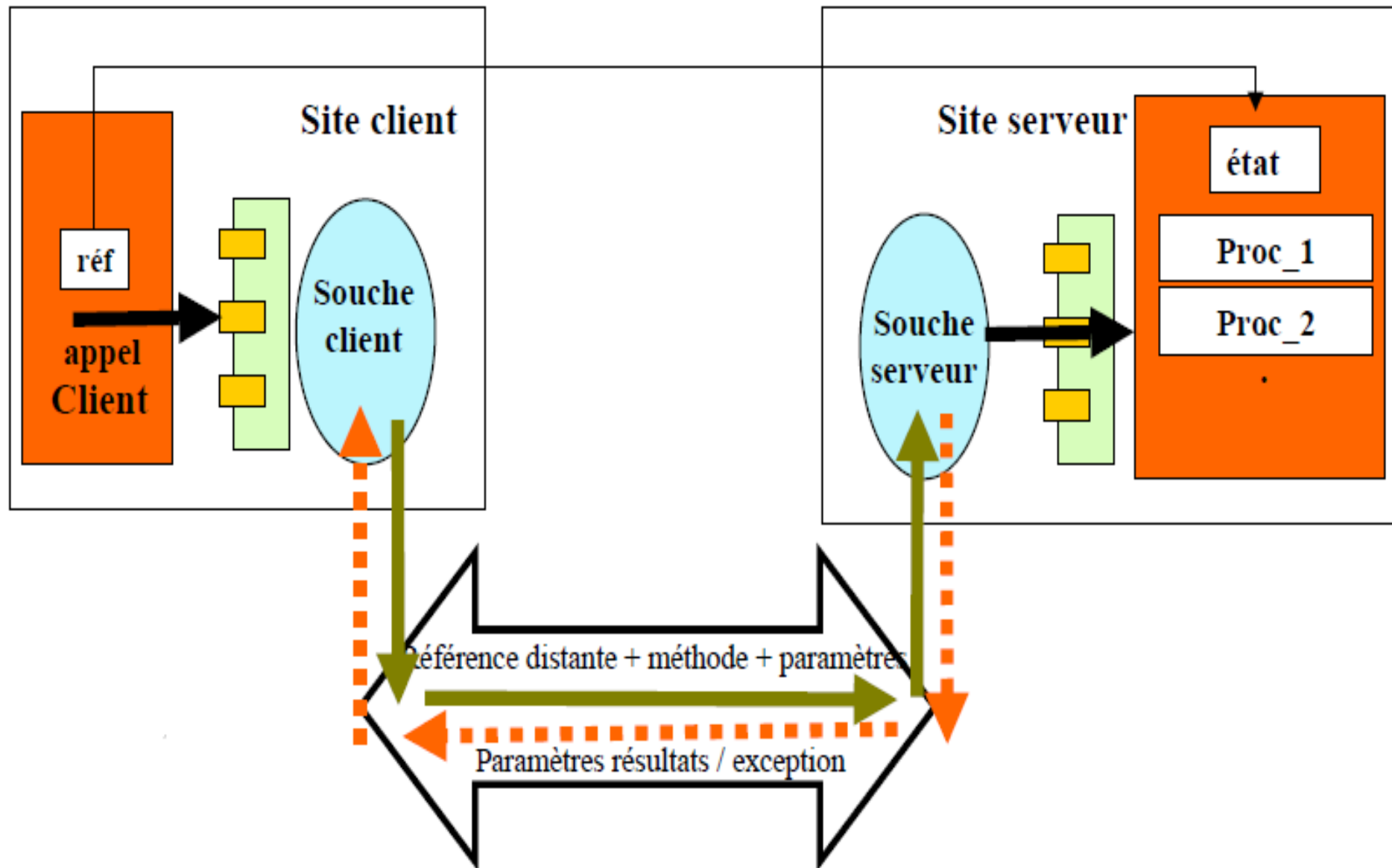




Souches: transformation d'un appel local en appel distant.

Objectif de génération automatique des souches connaissant le profil d 'appel de la procédure distante.

Très nombreuse terminologie dans ce cas : **Souches ("Stubs"), Talons, Squelettes ("Skeletons")...**





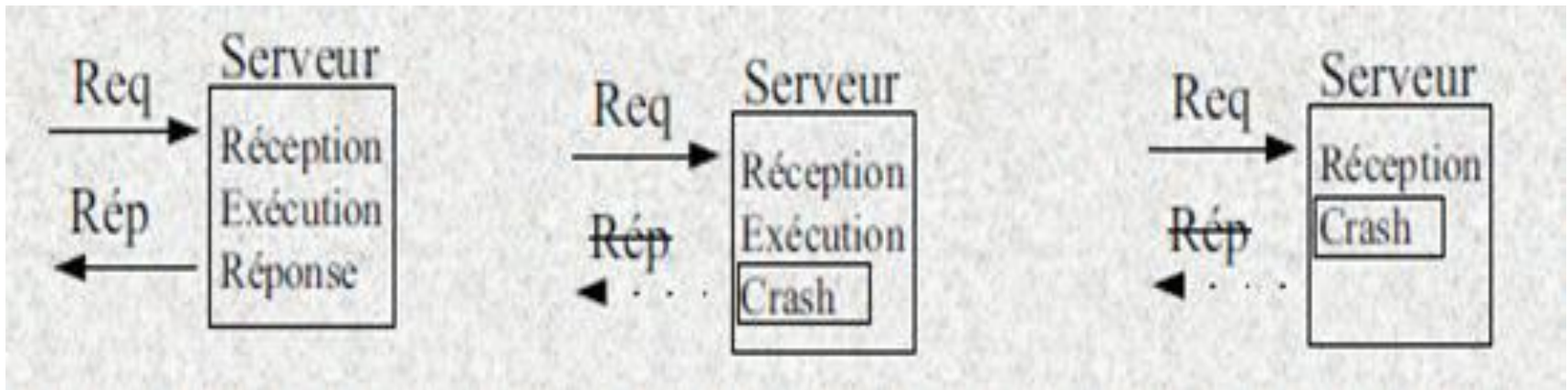
La souche client ("client stub")

- ✓ Procédure coté client qui reçoit l'appel en mode local
- ✓ Le transforme en appel distant
- ✓ En envoyant un message
- ✓ Reçoit le message contenant les résultats après l'exécution
- ✓ Retourne les résultats comme dans un retour de procédure.

La souche serveur ("server stub")

- ✓ Procédure coté serveur qui reçoit le message d'appel,
- ✓ Fait réaliser l'exécution sur le site serveur par la procédure serveur
- ✓ Récupère les résultats et retransmet les résultats par message.

Les crashes serveur sont plus durs à gérer, car on ne sais pas se qui s'est passé



Problème: On doit décider de ce qu'on attend du serveur

- **Sémantique d'au moins 1 :** Le serveur garanti qu'une opération est réalisée au moins 1 fois
- **Sémantique d'au plus 1 :** Le serveur garanti qu'une opération est exécutée au plus 1 fois



Sémantique exactement une fois

Définition théorique: Une seule exécution est effectuée et celle-ci réussit toujours.

Impossible au sens strict: des lors qu'une hypothèse de panne est formulée.

Comportement souhaité: le plus voisin possible de celui de l'appel de procédure en mode centralisé.

- ✓ Une suite de n tentatives est effectuée
- ✓ Avec sauvegarde état du client et du serveur avant chaque
- ✓ tentative. Pour une procédure déterministe



Cas d'une fonction quelconque (non idempotente): on ne doit pas l'exécuter deux fois.

Solution très répandue: garantir qu'on n'exécute pas deux fois une même procédure.

- ✓ Identification des requêtes + exécution effectuée une seule fois.

Si tout va bien le résultat est retourné à l'utilisateur.

- ✓ La requête ne sera plus exécutée (modulo le délai de garde de l'historique).

Si un problème est détecté Il est signalé à l'utilisateur pour qu'il puisse éventuellement statuer).

Aucune tentative de reprise n'est effectuée automatiquement. Elle est laissée entièrement à la charge du client.



- ❑ On se place dans le cas où chaque exécution du serveur est réalisée sans sauvegarde de l'état du serveur.
- ❑ L'exécution est lancée plusieurs fois si nécessaire (dans une certaine limite) pour obtenir une réponse correcte.
 - La procédure doit être idempotente.

Variante: La dernière de toutes

- ✓ On fait de plus l'hypothèse que les appels sont numérotés et traités séquentiellement.
- ✓ On est capable d'attribuer la réussite de l'exécution à la dernière de toutes les tentatives.



Problèmes :

1. Le client ne peut pas localiser le serveur 2 La requête cliente est perdue
2. La requête cliente est perdue
3. Le serveur crash
4. La réponse du serveur est perdue
5. Le client crashe

[1] Simple: on notifie le client

[2] On renvoie la requête



[4] On ne peut pas la distinguer du [3]. On ne sait donc pas si le serveur à exécuté la requête

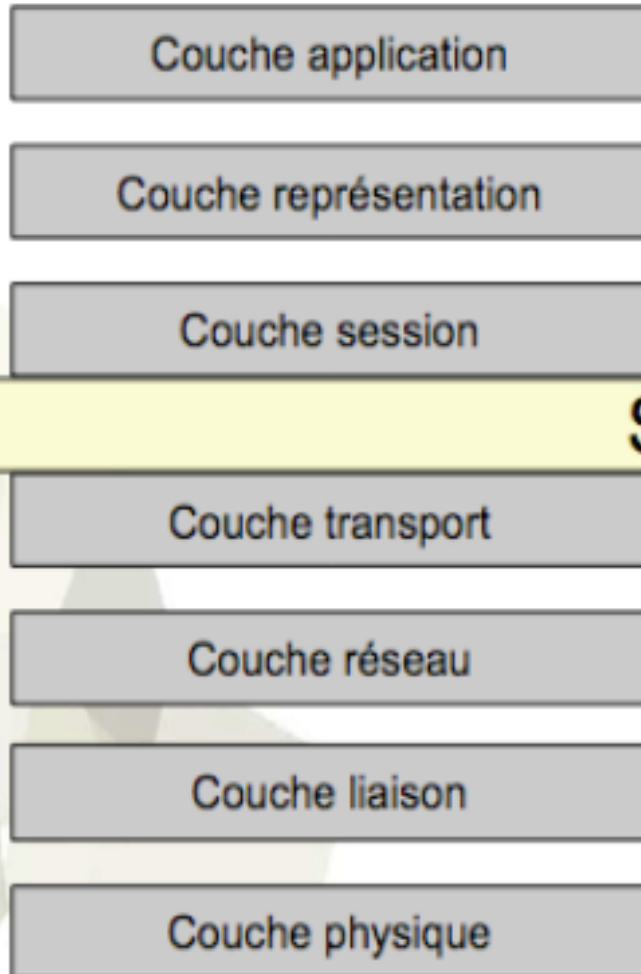
- Pas de solutions générale. Une première approche est d'avoir des opérations idempotentes

[5] **Problème** : le serveur réalise un calcul et bloque des ressources pour rien (calcul orphelin)

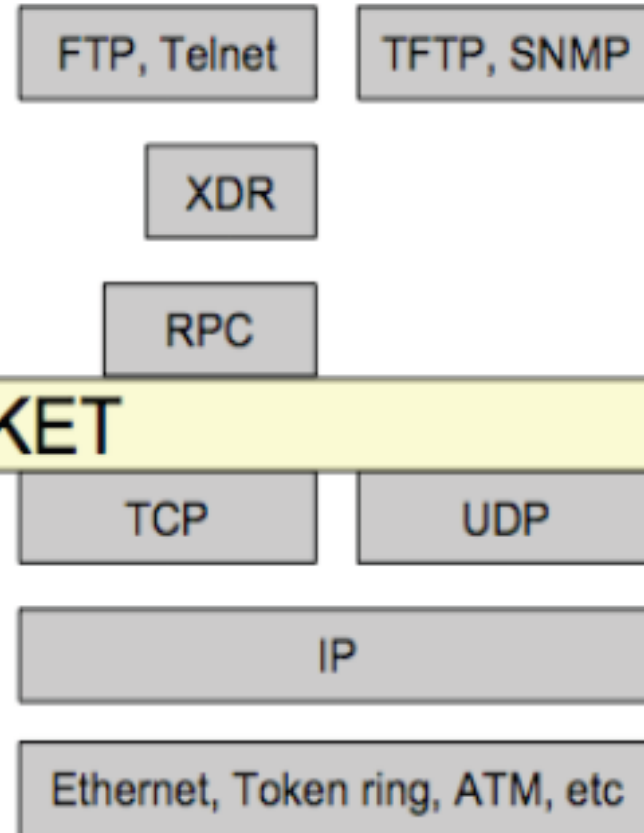
- Le processus orphelin est à la charge du client (reboot)
- Le processus orphelin est à la charge du réseau (Annonce d'une nouvelle époque)
- Le processus orphelin est à la charge du serveur (Le processus est borné par un time-out)

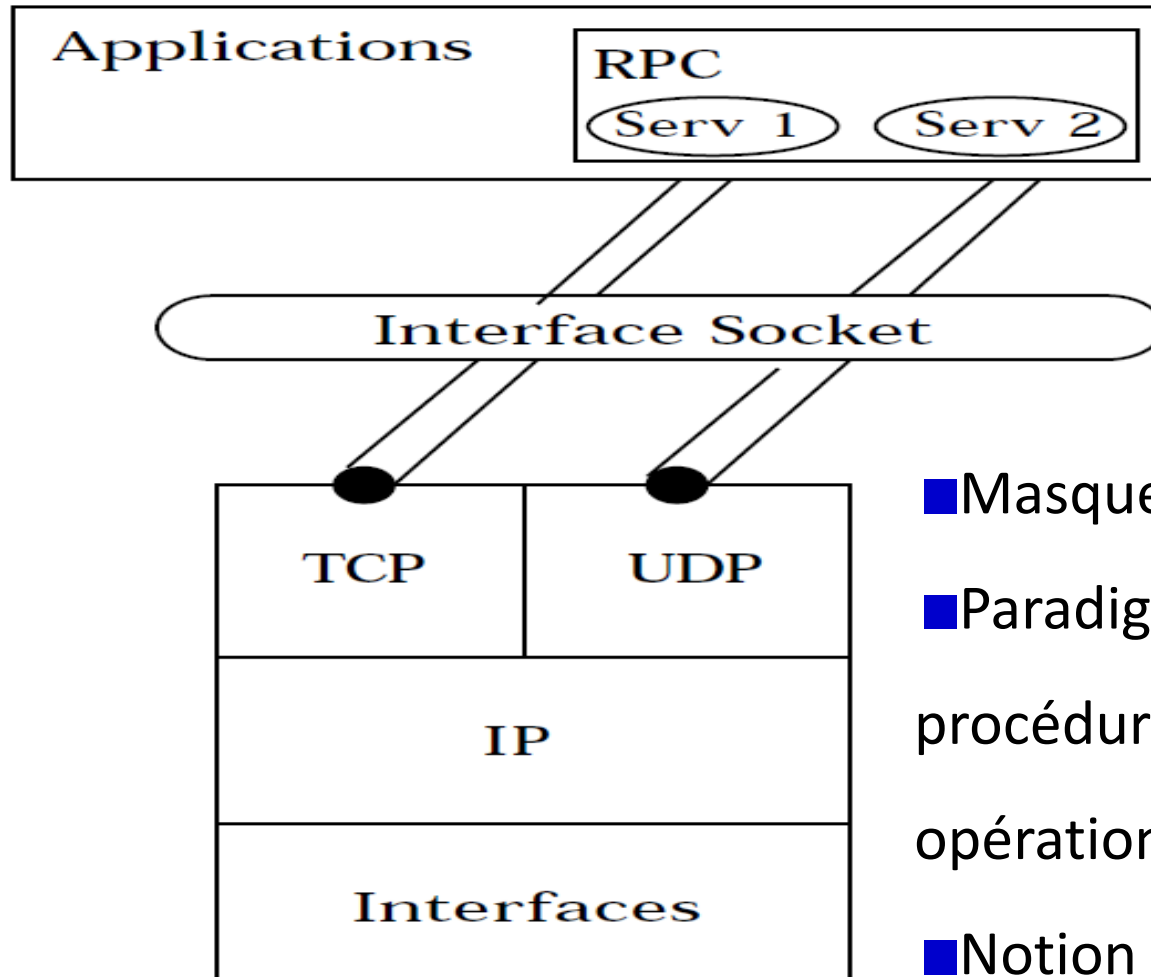


Modèle OSI



Modèle Internet





- Masquer l'interface socket
- Paradigme de l'appel de procédure en lieu et place des opérations send et receive
- Notion de souche/ subrogé (stub) client et serveur



- Transmission des paramètres
 - Multiples représentations
 - Passage par référence
- Localisation des serveurs
 - Enregistrement des services
 - Localisation d'un service
- Pannes
 - Défaillance du réseau
 - De client, de serveur



- La représentation des données
 - Multiples représentations
 - caractères : EBCDIC / ASCII
 - entier : complément à 1 ou à 2
 - représentation flottante
- taille des types
 - format "little endian"/ "big endian"
- Nécessité d'harmoniser la représentation
 - formats réseau et IEEE imposés
 - protocole XDR (eXternal Data Representation)



□ Langage XDR

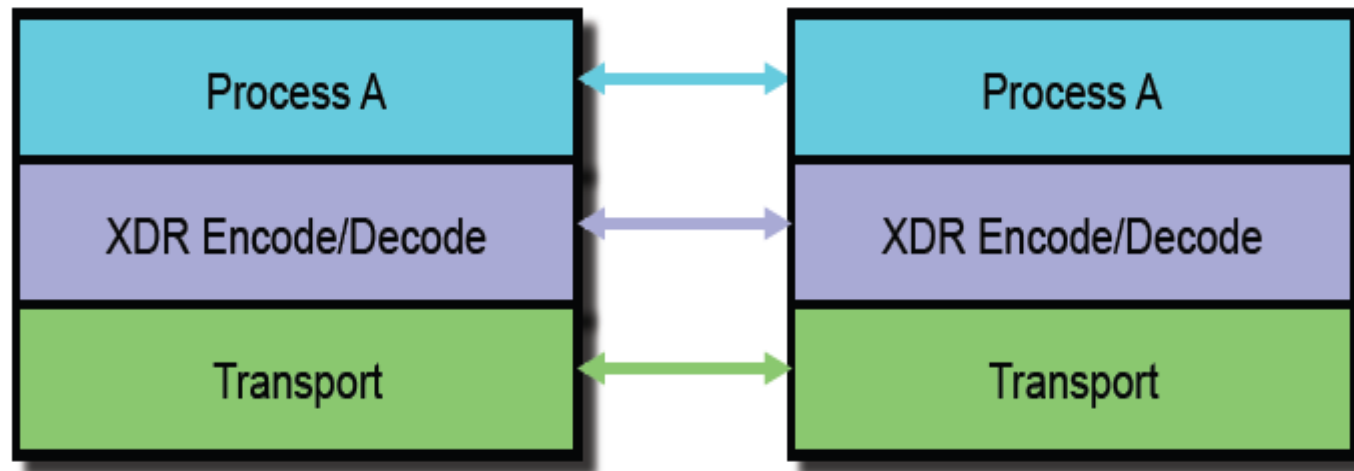
- Langage de description de la structure des données échangées lors du RPC
- Défini par le rfc 1014.
- Syntaxe issue de celle employée pour la description des données dans le langage C

□ Langage RPC

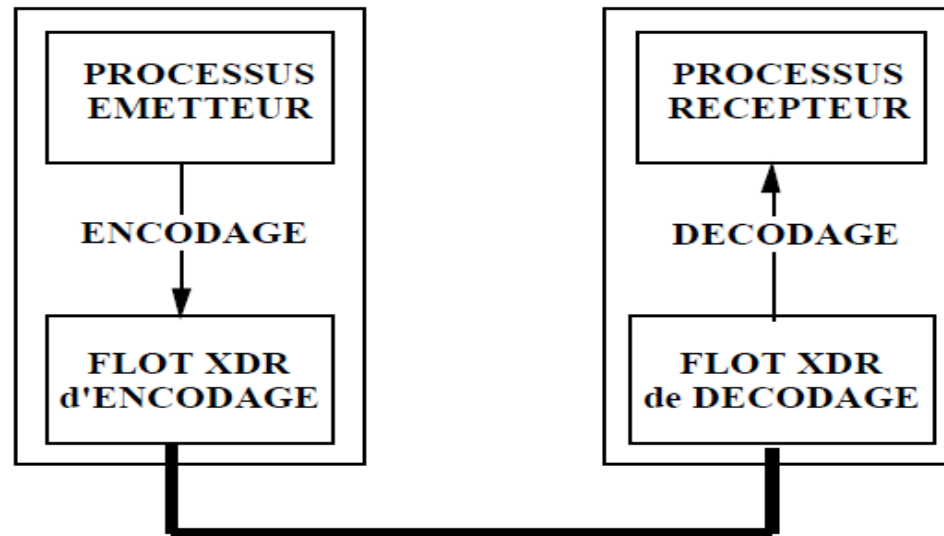
- Extension du Langage XDR, pour permettre la définition de procédure (programme)



- Masque l'hétérogénéité de représentation des données
- format standard, fonctions de transcodage
- Utilisation d'un protocole de sérialisation indépendant du client et du serveur
- Le talon client sérialise en XDR les arguments, envoie la requête au serveur, récupère le résultat et le "désérialise"
- Le talon serveur "désérialise" les arguments, exécute le code demandé, sérialise en XDR le résultat et l'envoie au client



- Encodage et décodage



⇒ **Flot XDR** : suite d'octets où les données sont représentées au format XDR

⇒ **Filtre XDR** : procédure qui encode et décode un certain type de données (entier, flottant, tableau...)

Un filtre XDR lit ou écrit des données dans un flot



Inconvénient du protocole XDR

- Mise en œuvre
- Deux machines de même type effectuent quand même les codages/décodages
- Eviter les codages / décodages inutiles
- Chaque machine utilise son format propre et précise celui-ci à son destinataire
- Le destinataire convertit les données dans son propre format si celui-ci est différent



- Un seul paramètre est échangé lors de l'appel (RPC call)
- Si l'application requiert l'échange de plusieurs paramètres ils doivent être regroupés au sein d'une seule structure de données.
- Un seul élément peut être échangé lors du retour (RPC reply) à travers la valeur de retour de la fonction



- **Le passage par référence**

⇒ **Deux formes de passages de paramètres**

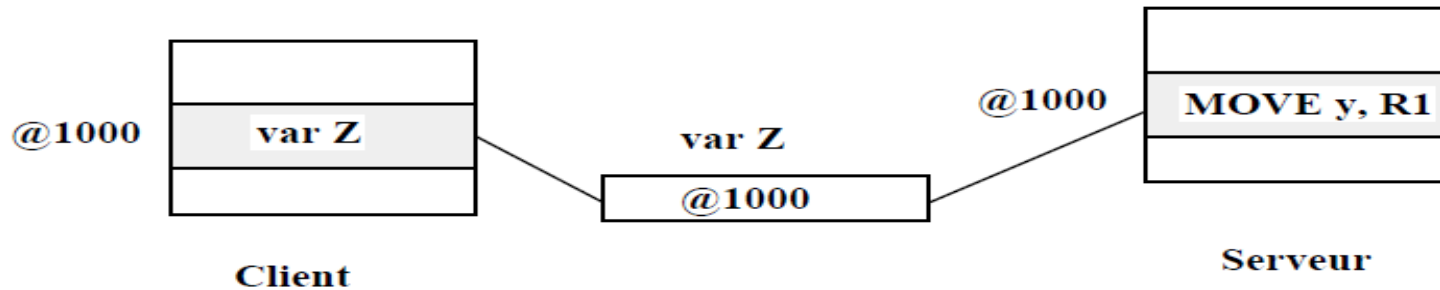
- **Par valeur**
- **Par adresse : quelle signification pour le processus distant ?**

⇒ **Solutions :**

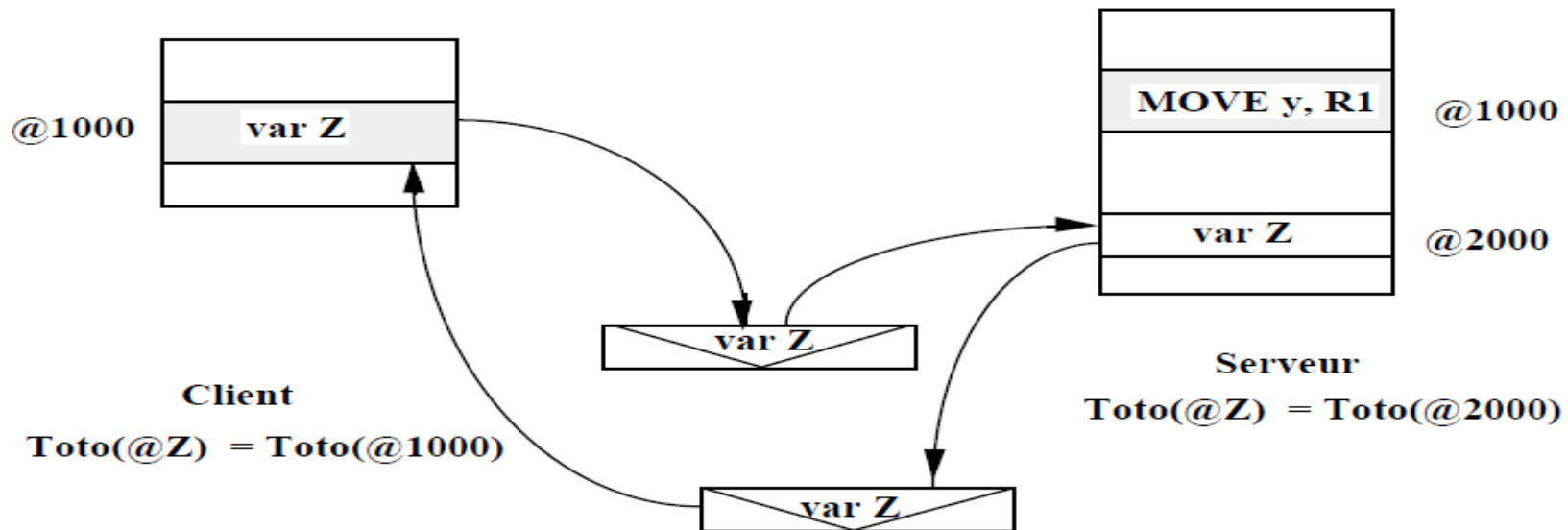
- **Interdire le passage d'adresse : transparence ?**
- **Utiliser le passage par valeur / référence**



- **Le passage par référence**



- **Le passage par valeur / référence**



- ⇒ **Nombreuses copies**
- ⇒ **Structures simples seulement**



- **Enregistrement des services**

⇒ Les services doivent se faire connaître

Spécification d'une procédure :

Nom_Du_Serveur : NFS

Numéro de Version : 2

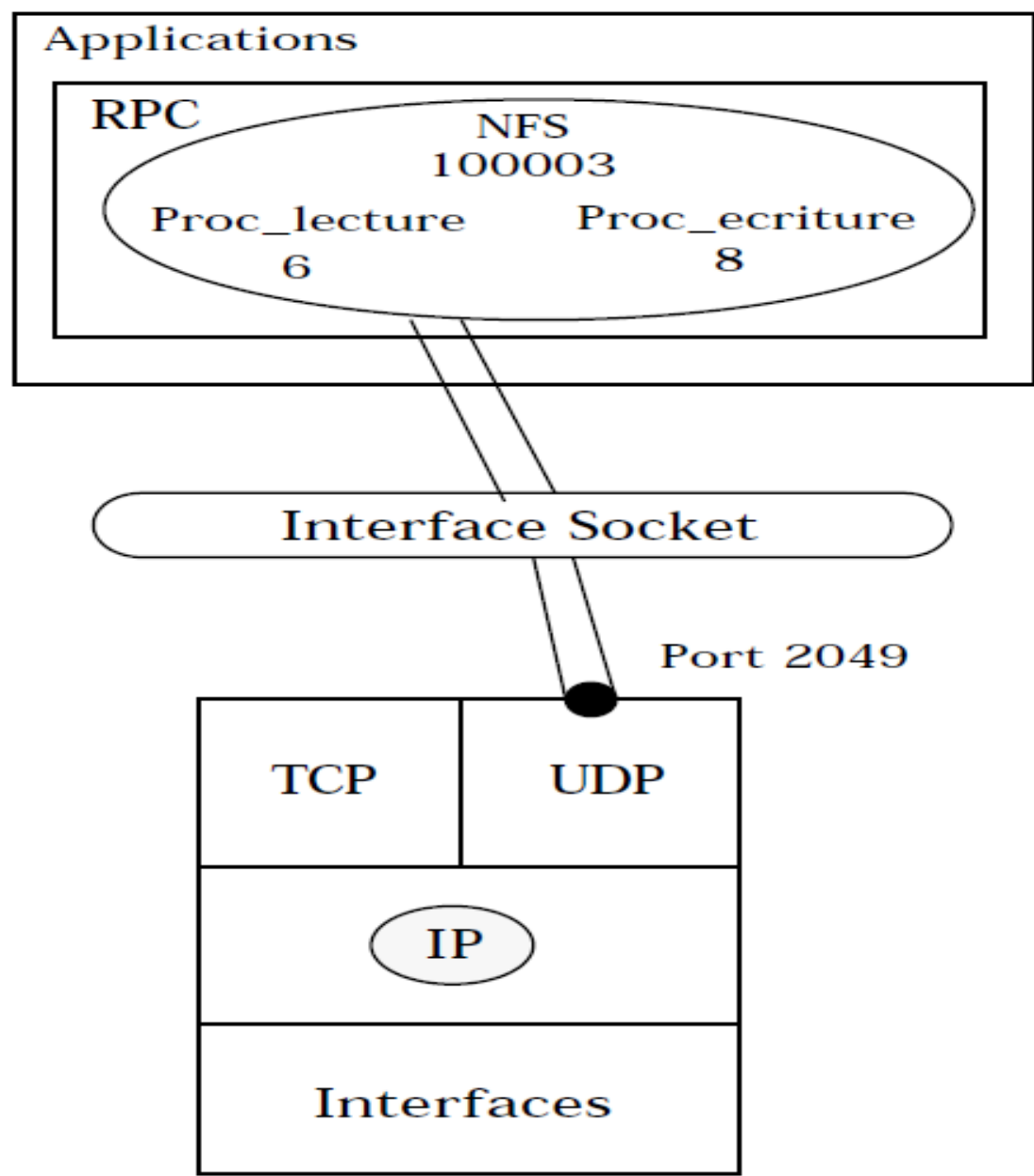
Identificateur unique 32 bits : 100003

Adresse : Handle : @IP, Port 2049

Authentication

Proc_Lecture : Entier : 6

Proc_Ecriture : 8



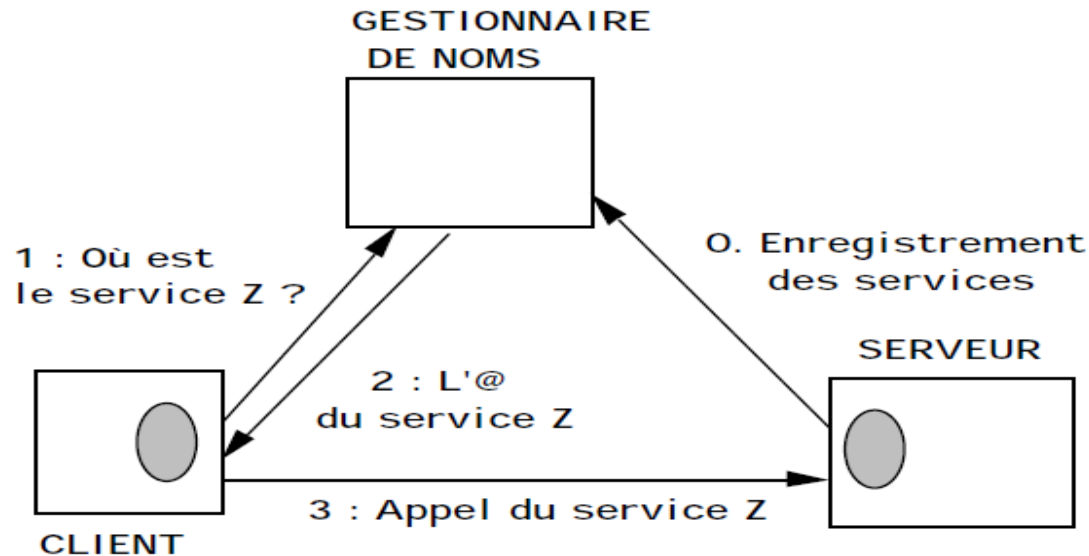


RPC : La localisation du serveur

- Localisation d'un service

Dynamiquement

- ⇒ Enregistrement auprès d'un gestionnaire de noms (Binder)
- ⇒ Le binder associe dynamiquement une @ (n° de port) à chacun des services





- **Nommage dynamique**

⇒ **Tolérance aux pannes** : le serveur peut régulièrement interroger les serveurs pour désenregistrer ceux qui sont indisponibles

⇒ **Protection** : le serveur de noms peut maintenir par service et par serveur une liste des utilisateurs acceptés

⇒ **Le serveur de noms peut devenir un goulot d'étranglement** : répartir la charge sur plusieurs serveurs disposant de la même interface mais attention au problèmes liés à la synchronisation dans la mise à jour des serveurs



■ Une procédure distante est identifiée de manière unique par un triplet :

- #program, #prog_version, #procedure

- Un programme regroupe un ensemble de procédures et possède une version

- plusieurs versions peuvent être disponibles simultanément

■ Certains numéros de programmes sont réservés à certains services :

- Réservé: 0x00000000 à 0x1FFFFFFFFFFF

- Public : 0x20000000 à 0x3FFFFFFFFFFF

- Semi-public : 0x40000000 à 0x5FFFFFFFFFFF

- Réservé : 0x60000000 à 0xFFFFFFFFFFFF

RPC : Identification des procédures



- Le portmapper permet de rediriger un client vers le numéro de port hébergeant le service

- Le portmapper est sur un numéro de port réservé : 111

- Les clients n'ont besoin de connaître que ce seul numéro de port/service

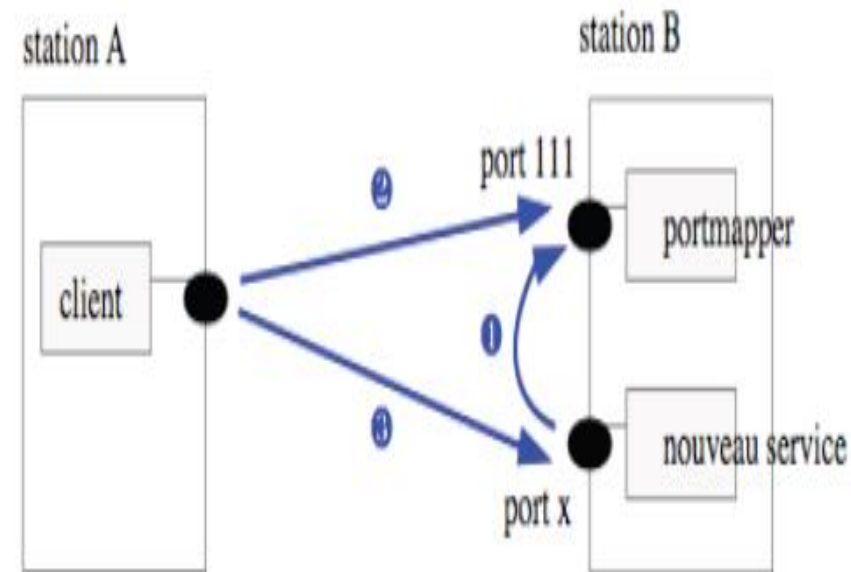
- Le portmapper est lui-même un service RPC

- description des procédures *portmap* en langage RPC

- description de la structure des messages

portmap en langage XDR

- `pmap_set()` = 1 ; enregistre un service
- `pmap_getport()` = 3 : retourne le numéro de port associé au service
- `pmap_rmtcall()` = 5 : appel d'une procédure distante



■ Système de distribution de services proposé par SUN au début des années 80 (utilisé pour le développement de NFS).

⇒ **Basé sur UDP ou TCP**

⇒ **Utilisation de XDR**

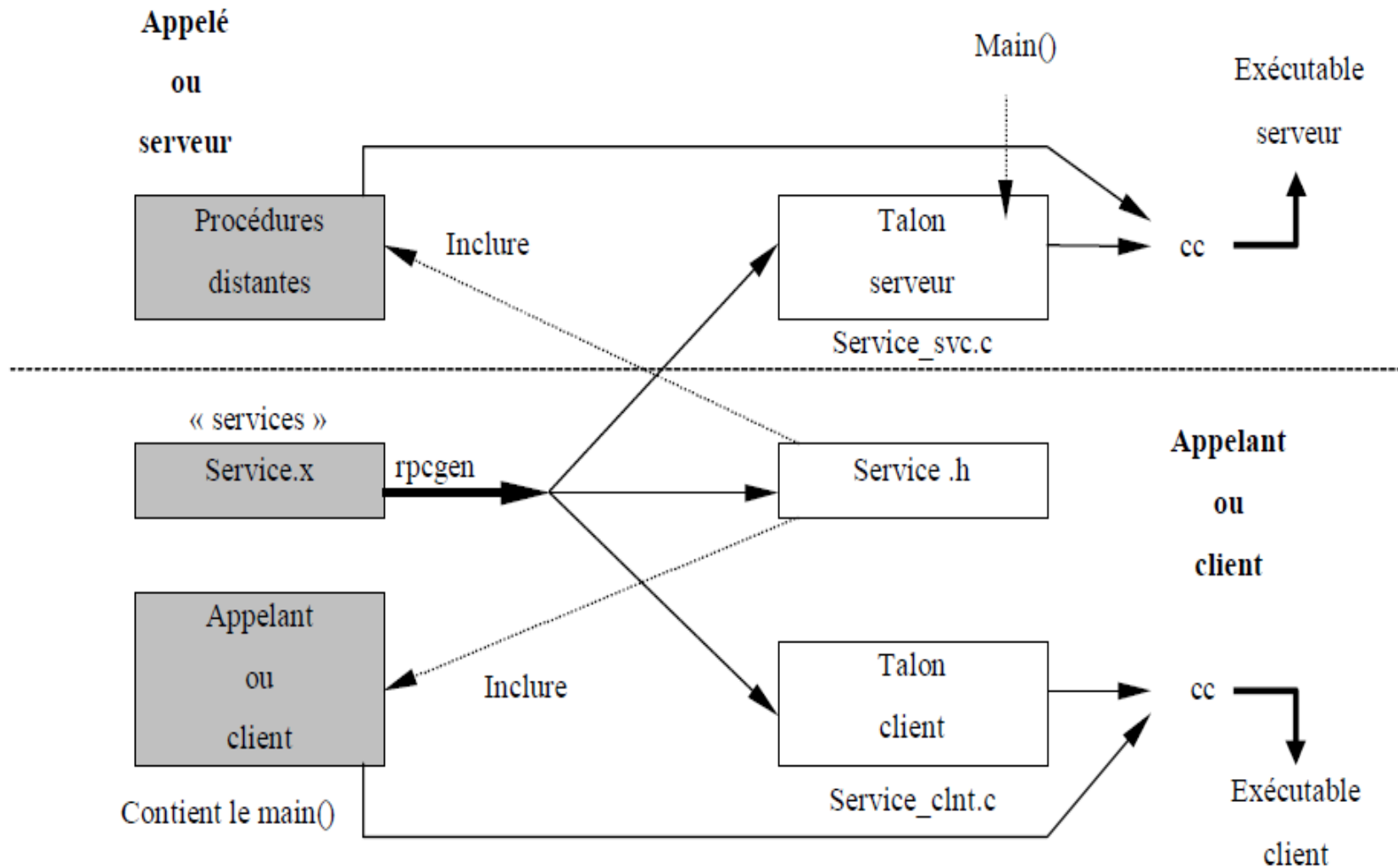
⇒ **Un seul argument et un seul résultat**

⇒ **Pannes :**

- **retransmission (UDP)**
- **sémantique au moins une fois**

⇒ **RPCGEN : Génération des souches client et serveur**

Le pré-compilateur RPCGEN





Donc, pour écrire une application répartie avec RPCGEN, il faut tout d'abord écrire un fichier qui décrit le **service**.

Dans la description du service, on trouve :

- le nom des procédures distantes,
- les types des paramètres.

On lui donne l'extension **.x**.

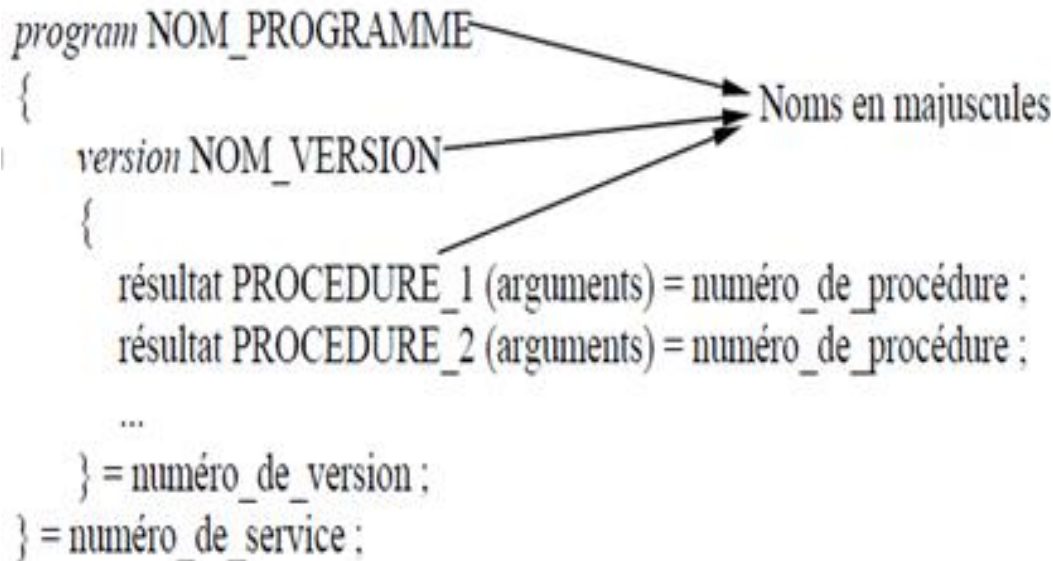
On passe ce fichier au pré-compilateur **rpcgen** qui génère les **talons**.

/* Description du « service » */

/* Les caractéristiques d'identification du *service distant* (programme, version(s), procédures distantes) sont écrites en langage : RPCL (RPC Language) */

/* **service.x** */

```
program NOM_PROGRAMME
{
    version NOM_VERSION
    {
        résultat PROCEDURE_1 (arguments) = numéro_de_procédure ;
        résultat PROCEDURE_2 (arguments) = numéro_de_procédure ;
        ...
    } = numéro_de_version ;
} = numéro_de_service ;
```



Noms en majuscules

* *numéro_de_procédure* : nombre > 0, différent pour chaque procédure.

* *numéro_de_version* : nombre > 0 permettant de différencier plusieurs versions d'un même service.

* *numéro_de_service* sur 32 bits (4 octets) :

- de 0x 0000 0000 à 0x 1fff ffff : réservés.
- de 0x 2000 0000 à 0x 3fff ffff : disponibles pour l'utilisateur.
- de 0x 4000 0000 à 0x 5fff ffff : temporaires.
- de 0x 6000 0000 à 0x 7fff ffff : réservés.

Exemple de protocole d'impression de message distant (par exemple, envoi d'un message dans un fichier distant)

```
program MESSAGE_PROG
{
    version MESSAGE_VERS
    {
        int PRINT_MESSAGE (string) = 1 ;
    } = 1 ;
} = 0x 2000 0001 ;
```



`service.x` → `service.h` qui devra être inclus dans :

- le fichier contenant les procédures distantes ;
- le `main()` du *client*.

→ `service_clnt.c`

→ `service_svc.c`

→ (éventuellement) des fonctions de codage/décodage XDR pour les paramètres et les résultats (`service_xdr.c`).

Ensuite il suffira de compiler séparément :

- les procédures distantes et le fichier généré `service_svc.c` ⇒ exécutable *serveur*.
- le `main()` du client et le fichier généré `service_clnt.c` ⇒ exécutable *client*.



- **XML-RPC** est un protocole RPC (Remote procedure call), une spécification simple et un ensemble de codes qui permettent à des processus s'exécutant dans des environnements différents de faire des appels de méthodes à travers un réseau.
- XML-RPC permet d'appeler une fonction sur un serveur distant à partir de n'importe quel système (Windows, Mac OS X, GNU/Linux) et avec n'importe quel langage de programmation. Le serveur est lui même sur n'importe quel système et est programmé dans n'importe quel langage.
- Cela permet de fournir un Service web utilisable par tout le monde sans restriction de système ou de langage.
- Les processus d'invocation à distance utilisent le protocole HTTP pour le transfert des données et la norme XML pour la structuration des données.



- XML-RPC est conçu pour permettre à des structures de données complexes d'être transmises, exécutées et renvoyées très facilement.
- XML-RPC est une alternatives aux Services Web WS, dont SOAP.
- XML-RPC was inspired by two earlier protocols. The first is an anonymous RPC protocol designed by Dave Winer and announced in an old DaveNet essay. (This is why XML-RPC servers are often installed under /RPC2.) The other, more important inspiration was an early draft of the SOAP protocol.



1. Gérard Florin, **L'appel de procédure distante ' RPC Remote Procedure Call '**, CNAM - Laboratoire CEDRIC .
2. Rabie Ben Atitallah, **OS Réseaux et Programmation Système - C5**,
3. N. Melab, **Génération de RPC à l'aide de RPCgen**, cours de Université de Lille1.
4. *George Coulouris, Jean Dollimore & Tim Kindberg, **Archive material from Edition 2 of Distributed Systems: Concepts and Design: SUN RPC, Edition 2, 1994.***
5. Nicolas Pouvesle and Kostya Kortchinsky, **Dissection des RPC Microsoft**, Actes du symposium SSTIC06, 2006.
6. Eric Kidd, **XML-RPC HOWTO**, Copyright © 2001 by Eric Kidd.